

Product Requirements Document

Return-Risk Scorer & Auto-Responder — AI Risk Manager Track, Razorpay AI Buildathon

Track	AI Risk Manager
Owner	[Your Name]
Status	Draft — Build in progress
Timeline	4-day build sprint
Dataset	E-Commerce Return Abuse Detection Dataset (Kaggle, 60k rows, 35 features)

1. Problem Statement

E-commerce merchants processing payments through Razorpay lose meaningful margin to return abuse — wardrobing (using and returning), policy abuse (excessive legitimate-looking returns), and outright fraudulent returns (empty box, wrong/stolen item). Manual review of every return does not scale past a small merchant, and blunt rule-based blocking (e.g. "flag after N returns") punishes legitimate customers and creates support escalations and churn.

There is currently no lightweight, explainable, and bounded way for a merchant to separate genuine returns from abusive ones at the point of request — with a decision trail that can be defended if a customer disputes the outcome.

"Stop the merchant losing money to fraud, returns and chargebacks. Build a working detector, verifier or auto-responder for one class of loss, with measured precision and recall on a held-out test set. The bar: honest metrics including false-positive cost. Strictly defense-only: anything offense-capable is disqualified." — Razorpay AI Buildathon, AI Risk Manager track brief

2. Goals & Non-Goals

Goals

- Score every incoming return for abuse risk across four classes: Legitimate, Policy Abuse, Wardrobing, Fraudulent Return.
- Make every decision explainable — surface the top reasons behind a risk score, not a black-box number.
- Keep every action bounded and reversible — never a silent, permanent auto-denial.
- Report honest, held-out precision/recall and an explicit false-positive cost, not cherry-picked examples.
- Log a complete audit trail for every decision made.

Non-Goals (out of scope for this build)

- No account suspension, blacklisting, or payment blocking — strictly defense-only, human-reversible actions.
- No real-money integration with live Razorpay APIs — evaluated on the batch/synthetic-replay dataset only.
- No multi-merchant configurability UI — a single policy profile is sufficient for this build.

3. Solution

A three-stage pipeline — Detector, Verifier, Auto-Responder — wrapped in a persistent audit log.

Stage	What it does	Technology
1. Detector	Multi-class ML model scores each return request across the four abuse classes and outputs a calibrated probability	LightGBM and XGBoost
2. Verifier	Policy engine converts the risk score into a bounded action using thresholds to minimize false-positive cost function	Thresholding
3. Auto-Responder	LLM agent takes the score + top reason codes and drafts (a) an internal message (explaining the flag) and (b) a customer-facing message	LLM API (e.g. OpenAI GPT-4)
4. Audit Trail	Every decision — input summary, score, action, rationale, timestamp — is logged and queryable.	JSON logs and SQL

Decision policy

Risk score band	Action	Rationale
Low	Auto-approve	Low cost of being wrong; approving fast preserves customer experience.
Medium	Hold for evidence	Ambiguous signal; auto-responder requests evidence, human reviews if unresolved.
High	Escalate to human	High confidence of abuse; still never auto-denied without a human check.
Insufficient signal	Route to manual review	The one explicitly handled failure case — the agent never guesses on incomplete data

4. Success Metrics

- **Precision / Recall / F1** per abuse class, measured on a held-out test split (never touched during training or threshold tuning).
- **Confusion matrix** across all four classes.
- **False-positive cost**: an explicit dollar estimate of wrongly holding/escalating a legitimate customer (support cost + churn risk) vs. the cost of a missed abuse case — this is what drives the threshold choice, not raw accuracy.
- **Throughput**: full batch of 50+ held-out records processed end-to-end with a reported match/resolution rate.
- **Exception rate**: share of cases routed to manual review via the insufficient-signal path, reported honestly rather than hidden.

5. Deliverables

Deliverable	Description
Public GitHub repo	Clean, reproducible code: data pipeline, model training, policy engine, agent layer, audit logger. One command to run.
README	Problem statement, architecture diagram, metrics table, the one gracefully-handled failure case, setup/run instructions.
Metrics report	Precision/recall/F1 per class, confusion matrix, false-positive cost calculation, on a true held-out split.
Audit trail sample	A set of real logged decisions (input → score → action → rationale → timestamp) shown in the repo or demo.
Architecture diagram	Detector → Verifier → Auto-Responder → Audit Trail, as included in this PRD.
Demo / dashboard (stretch)	Lightweight Streamlit or HTML view of batch results and example decisions.
5-minute pitch video	Problem → architecture → live demo (2-3 real examples) → metrics → the failure case → what's next.
Buildathon submission form	Submitted via the official Razorpay Google Form before the deadline.

6. Build Timeline (4 Days)

Day	Focus	End-of-day deliverable
Day 1	Data + baseline model: EDA, stratified split, first LightGBM classifier.	Working train → predict → evaluate pipeline.
Day 2	Feature engineering, class imbalance handling, false-positive cost function, decision policy, metrics table.	Trained model + policy table.
Day 3	Agent layer (case notes + customer messages), insufficient-signal path, audit-logged pipeline: one return in → decision + explanation.	End-to-end pipeline.
Day 4	Run on held-out batch, finalize metrics, write README, record pitch, submit.	Submitted repo + pitch video + form.

7. Compliance & Guardrails

- Strictly defense-only: the system flags, holds, and escalates — it never auto-denies, blocks accounts, or takes any offense-capable action.
- Every automated decision is reversible by a human reviewer.
- Metrics are reported on a genuinely held-out set; no cherry-picked examples used to represent overall performance.

- The insufficient-signal path ensures the model never forces a classification on incomplete data.